Elastic Certified Engineer Practice Exam

ENVIRONMENT OVERVIEW

Before starting the exam, ***please read through the following information carefully***:

- This exam consists of 10 tasks
- Each task is worth the same number of points
- Tasks are not graded as either "right" or "wrong" - you can earn partial credit on a task
- You can access the Elastic documentation using the shortcut on the Desktop of the exam environment.
- Do not ask the exam proctor if you can access the documentation - the answer is "yes", you can access any page on the elastic.co website. You can **not** browse the

documentation (or any other website) on your local computer - you can only use the web browser inside the exam environment.

- You are logged in to a Linux machine referred to as the **host** machine as a user named **elastic**
- Your exam environment has access to a collection of **servers**. The details of each server is listed below
- From the **host** machine, you can SSH to any **server** as the **elastic** user without a password. For example, you can use the following command to SSH onto **server1**:

```
$ ssh server1
```

- To login to the Kibana instances, the **username** is **elastic** and the **password** is **password**
- You can complete the tasks in any order, so feel free to read all of the tasks first if that helps you plan your time better

THE CLUSTERS

The **servers** are configured to be **two** Elasticsearch clusters with the following details:

Details of `cluster1`

A 3-node Elasticsearch cluster with a Kibana instance:

Server Name	IP address	HTTP Port	Transport Port	Comments
server1	172.18.0.11	9201	9301	An Elasticsearch node in **cluster1**
server2	172.18.0.12	9202	9302	An Elasticsearch node in **cluster1**

Server Name	IP address	HTTP Port	Transport Port	Comments
server3	172.18.0.13	9203	9303	An Elasticsearch node in **cluster1**
kibana1	172.18.0.15	5601	n/a	Kibana is configured to connect to the cluster and can be accessed by clicking the **Kibana1** button in Strigo

---

Details of `cluster2`

A 1-node Elasticsearch cluster with a Kibana instance:

Server name	IP address	HTTP Port	Transport Port	Comments
server4	172.18.0.14	9204	9304	An Elasticsearch node in **cluster2**
kibana2	172.18.0.16	5602	n/a	Kibana is configured to connect to the cluster and can be accessed by clicking the **Kibana2** button in Strigo

---

Exam Tasks

1. Create a user on **cluster1** satisfying the following requirements:
    - the user is named **task1**
    - the user has full cluster privileges
    - the user has read-only access to all indices, except for an index called **task1** where the user has full privileges
    - the user has full access to all features in Kibana in the Default space
    - the user's full name is "Elastic Search"
    - the user's email address is "elastic@search.com"
    - the user's password is "youknowforsearch"

    Show answer

    The simplest way to complete this task is to use Kibana to create this user and the user's role.

    Begin by creating the role. Go to **Stack Management** then **Roles** under **Security**.

    Create a new role. Call it **task1_role**.

    Under **Cluster privileges** select `all`. Leave **Run as privileges** blank.

    Under **Index privileges** enter `*` for indices and `read` for privileges. Add another index privilege. Enter `task1` for indices and `all` for privileges.

    Add Kibana privileges. Select the **Default** space. Set **Privileges for all features** to **All**.

    Create this role.

Go to **Users**. Fill in all the credentials and select the Role **task1_role**. Create this user.

2. Find the number of unique authors for each calendar year of **blogs** on **cluster1** with the phrase "open source" in that order in the `content` field. Use the `authors.uid` field to find unique authors. Do not return any hits.

   Show answer

```
GET blogs/_search
{
  "size": 0,
  "query": {
    "match_phrase": {
      "content": "open source"
    }
  },
"aggs": {
  "yearly": {
    "date_histogram": {
      "field": "publish_date",
      "calendar_interval": "year"
    },
    "aggs": {
      "unique_authors": {
        "cardinality": {
          "field": "authors.uid.keyword"
        }
      }
    }
  }
 }
}
```

3. Reindex the **blogs** index on **cluster1** into a new index called **task3** on **cluster1** satisfying the following requirements:
   - all string fields are mapped as `keyword` only, except for the fields `content`, `title`, `authors.full_name` which are type `text`
   - the `authors.full_name` field uses the standard analyzer

- the `content` field uses a custom analyzer called `content_analyzer` that uses the `html_strip` character filter, the `standard` tokenizer, and the `lowercase` token filter
- the `content` field has a multi-field called `eng` using the `english` analyzer
- the `title` field uses the standard analyzer and has a multi-field called `search` of type `search_as_you_type`

Show answer

First set up the index with the custom analyzer `content_analyzer` and the mapping. You can let Elasticsearch determine the mapping for any unspecified fields.

```
PUT task3
{
  "settings": {
    "analysis": {
      "analyzer": {
        "content_analyzer": {
          "char_filter": ["html_strip"],
          "tokenizer": "standard",
          "filter": ["lowercase"]
        }
      }
    }
  },
  "mappings": {
    "dynamic_templates": [
      {
        "map-strings": {
          "match_mapping_type": "string",
          "match": "*",
          "mapping": {
            "type": "keyword"
          }
        }
      }
    ],
    "properties": {
      "authors": {
        "properties": {
          "full_name": {
            "type": "text"
          }
        }
      },
      "content": {
        "type": "text",
        "analyzer": "content_analyzer",
        "fields": {
```

```
            "eng": {
              "type": "text",
              "analyzer": "english"
            }
          }
        },
        "title": {
          "type": "text",
          "fields": {
            "search": {
              "type": "search_as_you_type"
            }
          }
        }
      }
    }
  }
```

Then reindex into the new index.

```
POST _reindex
{
    "source": {
        "index": "blogs"
    },
    "dest": {
        "index": "task3"
    }
}
```

4. **cluster1** has an index called **web_traffic**. Here is an example of a log from one of those visits:

```
{
  "@timestamp": "2021-03-21T19:25:05.000-06:00",
  "bytes_sent": 26774,
  "content_type": "text/html; charset=utf-8",
  "geoip_location_lat": 39.1029,
  "geoip_location_lon": -94.5713,
  "is_https": true,
  "request": "/blog/introducing-elastic-endpoint-security",
  "response": 200,
  "runtime_ms": 191,
  "user_Agent": "Mozilla/5.0 (compatible; MJ12bot/v1.4.8; http://mj12bot.com/)",
  "verb": "GET"
}
```

Reindex the **web_traffic** index into a new index on **cluster1** named **task4** satisfying the following requirements:

- remove the field `is_https`
- rename the field `geoip_location_lat` to `geo.location.lat`

- rename the field `geoip_location_lon` to `geo.location.lon`
- use the `user_agent` processor on the field `user_Agent`
- remove the field `user_Agent`
- the `geo.location` object is stored as type `geo_point`
- all other fields are type `keyword` except for `runtime_ms`,`bytes_sent` which are type `long` and `@timestamp` which is type `date`
- the index has 6 primary shards and 0 replicas

Show answer

In Kibana go to **Stack Management**, then click **Ingest Node Pipelines**. Click **Create new pipeline**, then add each processor. If you want to bypass the UI and just define the pipeline using an HTTP request, copy-and-paste the following into the **Console**:

```
PUT _ingest/pipeline/web_traffic_pipeline
{
  "processors": [
    {
      "remove": {
        "field": "is_https",
        "ignore_missing": true
      }
    },
    {
      "rename": {
        "field": "geoip_location_lat",
        "target_field": "geo.location.lat",
        "ignore_missing": true
      }
    },
    {
      "rename": {
        "field": "geoip_location_lon",
        "target_field": "geo.location.lon",
        "ignore_missing": true
      }
    },
    {
      "user_agent": {
        "field": "user_Agent",
        "ignore_missing": true
      }
    },
    {
```

```
      "remove": {
        "field": "user_Agent",
        "ignore_missing": true
      }
    }
  ]
}
```

Use the following settings and mapping for `task4` (copy-and-paste it into your **PUT** request):

```
  "settings": {
    "number_of_shards": 6,
    "number_of_replicas": 0
  },
  "mappings": {
    "properties": {
      "geo": {
        "properties": {
          "location": {
            "type": "geo_point"
          }
        }
      },
      "request": {
        "type": "keyword"
      },
      "status_code": {
        "type": "keyword"
      },
      "url": {
        "type": "keyword",
      },
      "user_agent": {
        "properties": {
          "device": {
            "properties": {
              "name": {
                "type": "keyword"
              }
            }
          },
          "name": {
            "type": "keyword"
          },
          "original": {
            "type": "keyword"
          },
          "version": {
            "type": "keyword"
          }
        }
      }
    }
  }
```

Then reindex `web_traffic` into `task4`.

```
POST _reindex
{
    "source": {
        "index": "web_traffic"
    },
```

```
    "dest": {
        "index": "task4",
        "pipeline": "web_traffic_pipeline"
    }
}
```

5. Reindex **blogs** into a new index on **cluster1** named **task5** where the `authors` object is of type `nested`. Write a query on `nested_authors` that searches for blogs where `authors.first_name` is **"david"** and the `authors.job_title` contains the term **"engineer"**.

   Show answer

   First create the new index with a mapping where the `authors` field is of type `nested`.

```
PUT task5
{
  "mappings": {
    "properties": {
      "authors": {
        "type": "nested"
      }
    }
  }
}
```

   Reindex the `blogs` documents into `task5`.

```
POST _reindex
{
    "source": {
        "index": "blogs"
    },
    "dest": {
        "index": "task5"
    }
}
```

   Now write the nested query.

```
GET task5/_search
{
  "query": {
    "nested": {
      "path": "authors",
      "query": {
        "bool": {
          "must": [
            {
              "match": {
                "authors.full_name": "david"
              }
            },
```

```
            {
              "match": {
                "authors.job_title": "engineer"
              }
            }
          ]
        }
      }
    }
  }
}
```

You should get 22 hits, and each of them should contain an author named David with Engineer in their job title.

6. Reindex the **blogs** index on **cluster1** into a new index on **cluster1** named `task6` satisfying the following requirements:
    - enrich data from the `categories` index by adding the `title` for `uid` values that correspond with `category` values
    - remove the old `category` field
    - make sure to deal with documents that do not have a value for the `category` field The new index `task6` will have documents that look like the following:

```
{
  "locale" : "en-us",
  "title" : "Second Annual Elastic{ON} Women's Breakfast: Now with Leadership
Panel",
  "content" : """...""",
  "url" : "/blog/second-annual-elastic-on-womens-breakfast-now-with-
leadership-panel",
  "tags" : { ... },
  "categories" : {
    "uid" : "bltc253e0851420b088",
    "title" : "Culture"
  },
  "publish_date" : "2017-02-06T08:00:00-0600",
  "authors" : [ ... ]
}
```

Show answer

First set up the enrich policy.

```
PUT /_enrich/policy/categories_policy
{
```

```
  "match": {
    "indices": "categories",
    "match_field": "uid",
    "enrich_fields": ["title"]
  }
}
```

Now execute the policy.

```
POST /_enrich/policy/categories_policy/_execute
```

Use the Ingest Node Pipeline UI to define a pipeline in Kibana, or use the API directly in the Console:

```
PUT _ingest/pipeline/categories_pipeline
{
  "processors": [
    {
      "enrich": {
        "field": "category",
        "policy_name": "categories_policy",
        "target_field": "categories",
        "ignore_missing": true
      }
    },
    {
      "remove": {
        "field": "category",
        "ignore_missing": true
      }
    }
  ]
}
```

Now use the pipeline in a reindex request.

```
POST _reindex
{
  "source": {
    "index": "blogs"
  },
  "dest": {
    "index": "task6",
    "pipeline": "categories_pipeline"
  }
}
```

Verify that the reindex request was successful. The following request should return 537 hits.

```
GET task6/_search
{
  "query": {
    "match": {
      "categories.title": "news"
    }
  }
}
```

7. Replicate the **blogs** index on `cluster2` to an index on **cluster1** named **ccr_blogs**.

   Show answer

   You need a license to use cross-cluster replication. Go to **Stack Management -> License Management** and start a 30-day trial of the license. Also start the trial on `cluster2`.

   On **Kibana1**, go to **Stack Management**, then select **Remote Cluster**. Add a new remote cluster with the following properties:

   - Name: `cluster2`
   - Seed nodes: `server4:9304`
   - Node connections: 1

   Go to the **Cross-Cluster Replication** page in **Stack Management** and select **Create a follower index**.

   Configure CCR so that the `blogs` index on `cluster2` is replicated to `cluster1` in a new index named `ccr_blogs`.

   You can also complete this task by running the following command in the **Console** of **Kibana1**:

```
PUT /ccr_blogs/_ccr/follow
{
  "remote_cluster": "cluster2",
  "leader_index": "blogs",
  "max_read_request_operation_count": 5120,
  "max_outstanding_read_requests": 12,
  "max_read_request_size": "32mb",
  "max_write_request_operation_count": 5120,
  "max_write_request_size": "9223372036854775807b",
  "max_outstanding_write_requests": 9,
  "max_write_buffer_count": 2147483647,
  "max_write_buffer_size": "512mb",
  "max_retry_delay": "500ms",
  "read_poll_timeout": "1m"
}
```

   Your new follower index is paused initially. Start the replication process by clicking the **Manage** button in the bottom-right corner of the screen you are viewing; or if you

already closed that window, the same options appear by clicking the three dots under **Actions** on the **Cross-Cluster Replication** UI page.

Run the following query in the **Console** of `cluster1`. You should get a count total of 7:

```
GET /ccr_blogs/_count
```

8. Create a data stream on **cluster1** that satisfies the following requirements:
    - the index is in the hot phase for 2 minutes
    - when the index rolls over, it immediately moves to the warm phase and becomes read-only
    - after 3 minutes in the warm phase, the index moves to the cold phase where it will remain indefinitely
    - the data stream is called `my.metrics-system-dev`
    - the index template uses the `metrics-mapping` component template

    Show answer

    In Kibana, go to **Stack Management -> Index Lifecycle Policies**.

    Click **Create policy** to start a new policy.

    Name the policy `my-metrics-policy`.

    In the **Hot** phase, expand the **Advanced settings**. Turn off **Use recommended defaults**. Set the rollover maximum age to 2 minutes and also delete the value for maximum size. Leave the remaining defaults.

    Activate the **Warm** phase. Set the **Move data into phase when** to 0. Enable Read only. Leave the remaining defaults.

Activate the **Cold** phase. Set the **Move data into phase when** to 3 minutes.

Click **Save policy**.

The policy should look like the following:

```
PUT _ilm/policy/my-metrics-policy
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": {
            "max_age": "2m"
          },
          "set_priority": {
            "priority": 100
          }
        }
      },
      "warm": {
        "min_age": "0m",
        "actions": {
          "readonly": {}
        }
      },
      "cold": {
        "min_age": "3m",
        "actions": {
          "set_priority": {
            "priority": 0
          }
        }
      }
    }
  }
}
```

In **Index Management**, go to **Index Templates**. Click the **Create template** button.

The name of the index template should be `my-metric-template`.

The index pattern should use the data stream name convention, in this case `my.metrics-*-*`.

Enable `Create data stream`. Click **Next**.

Select the `metrics-mappings` component template by clicking the "+" button.

No other changes are needed, click **Next** until you get to the **Review** page. Click **Create template**.

Go back to **Index Lifecycle Policies** and search for `my-metrics-policy` using the search bar.

Click **Actions** then **Add policy to index template**.

Select `my-metrics-template`. Leave the **Alias for rollover index** blank.

Finally, you have to manually start the stream.

```
PUT _data_stream/my.metrics-system-dev
```

Now if you go to **Data Streams** in **Index Management**, you should see `my.metrics-system-dev`.

9. Register a new repository on **cluster1** that satisfies the following requirements, keeping in mind that **cluster1** already has `path.repo` set to `/var/tmp/repo`, which is a shared file system on all nodes:
    - the name of the repository is `snap-repo`
    - it is a shared file system that maps to the `/var/tmp/repo/search` folder

    There is already a snapshot in `snap-repo` called `moreblogs_snapshot`. It contains only one index called `moreblogs`. Write a request to search `moreblogs` for blogs that have a `tags.product` with `elasticsearch`.

    Show answer

    You need a license to use searchable snapshots. Go to **Stack Management -> License Management** and start a 30-day trial of the license if you haven't already.

    Click the **Repositories** tab.

    Click **Register a repository** button.

Name your repository `snap-repo`.

Select **Shared file system** as the type. Click **Next**.

Set the **File system location** to `/var/tmp/repo/search` and click **Register**.

You can verify the repository by clicking on **Verify**. You should see three nodes: `node1`,`node2`,`node3`.

Now mount the snapshotted index. Go to **Console** in **Dev Tools**.

```
POST _snapshot/snap-repo/moreblogs_snapshot/_mount
{
  "index": "moreblogs"
}
```

You can now search `moreblogs` as you would any other index.

```
GET moreblogs/_search
{
  "query": {
    "match": {
      "tags.product": "elasticsearch"
    }
  }
}
```

You should get 22 hits.

10. Write a query on the index **blogs** index on **cluster1** that satisfies the following requirements:
    - find all blogs that write about `certified engineer` in the `content` field
    - boost any blogs that contain the phrase `certified engineer` in the `content` field in that order
    - find only blogs that were published in the last 2 years
    - sort the hits first by the `_score` in descending order, then `_doc` in ascending order
    - retrieve the **3rd page** where each page contains 5 documents

- highlight any occurrence of `certified engineer` in the `content` field

Show answer

```
GET blogs/_search
{
  "from": 10,
  "size": 5,
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "content": "certified engineer"
          }
        }
      ],
      "should": [
        {
          "match_phrase": {
            "content": "certified engineer"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "publish_date": {
              "gte": "now/d-2y"
            }
          }
        }
      ]
    }
  },
  "sort": ["_score","_doc"],
  "highlight": {
    "fields": {
      "content": {}
    }
  }
}
```

© Elasticsearch BV 2015-2021. All rights reserved. Decompiling, copying, publishing and/or distribution without written consent of Elasticsearch BV is strictly prohibited.
Made with Material for MkDocs